Numpy: A Python Library for ND-Arrays and Matrices

Python course for Armed Forces Personnel
Indian Institute of Space Science and Technology.

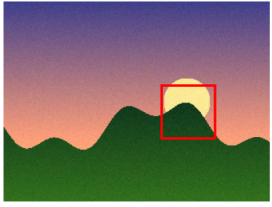
6th October 2026

Motivation

- A programming language should give us the ability to construct higher level abstractions and work with them
- We should be able to think about numerical lists, arrays, and matrices - without worrying about how it is internally represented.
- Why numerical lists, arrays, and matrices - numerical data from sensors, images, internal representations of models are naturally represented like this.
- Recall: we have variables, lists, dictionaries - but it becomes clumsy to use them

An example: an image inside a computer

the image

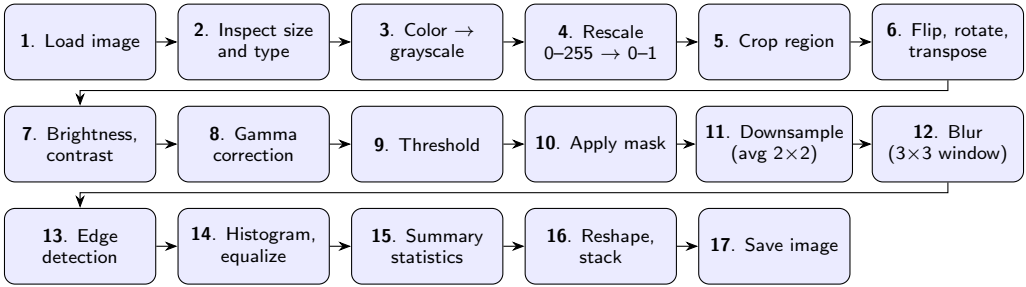


red box = 8x8 grid of numbers

134	140	230	222	228	229	235	134
148	231	229	230	229	233	230	152
149	235	233	230	231	224	228	233
152	231	232	41	44	231	234	228
168	43	49	46	43	45	232	165
47	46	47	50	41	51	42	166
47	57	46	49	54	53	55	51
52	51	52	51	57	51	47	60

- Grayscale image: 2D grid, 0 (black) to 255 (white)
- Color image: three grids, shape `(height, width, 3)`
- 12-megapixel photo $\approx$ 36 million numbers
- Brighten, crop, blur, find edges = arithmetic on the grid

An example: image processing



An example of image processing (note that all these steps may not be required, say for a task such as identifying the sun).

What pattern do we see?

Verbs: *slice*, *flip*, *scale*, *compare*, *sum*, *average*, *slide a window*, *reshape*, *stack*.

1. The same operation could be done on every element of the matrix/array
2. We might need to concentrate on a sub-block
3. Do operations on a number of elements and obtain scalar or smaller-dimensional output.

Such operations appear in nearly all tasks related to working with matrices and arrays.

Pure Python: gamma correction

Question: How would you do step 8 (gamma) in pure Python?

```
result = []
for row in image:
    new_row = []
    for pixel in row:
        new_row.append(pixel ** 0.7)
    result.append(new_row)
```

Problems:

- Verbose: nested loops for a one-line idea
- Slow: interpreter-speed loops over millions of elements
- Memory-hungry: every number is a full Python object
- A 3×3 blur needs four nested loops

We need "do this to the whole array" in one line, and fast.

NumPy

- `ndarray`: fixed-type, contiguous memory, with a shape
- Compiled C inside ⇒ fast
- **Vectorization**: operate on whole arrays
- **Broadcasting**: sensible rules for mixed shapes
- Foundation of pandas, SciPy, scikit-learn, matplotlib, PyTorch

```
import numpy as np
a = np.arange(1000000)
[v*2 for v in range(1000000)]  # list
a * 2                          # NumPy
```

Measured:

```
pure Python:   69.13 ms
NumPy:          0.51 ms
speedup:         135x
```

How to work with arrays in NumPy

First we will ask Python to let us use Numpy library.

```
import numpy as np
```

```
from PIL import Image
from numpy.lib.stride_tricks import sliding_window_view
```

Steps 1–2: Load and inspect

Concept: `np.asarray`; attributes `.shape` `.dtype` `.ndim` `.size`

```
img = np.asarray(Image.open("photo.jpg"))
print(img.shape)
print(img.dtype)   # uint8
print(img.ndim)
print(img.size)
```

```
shape: (240, 320, 3)
dtype: uint8
ndim:  3
size:  230400
```

img (240, 320, 3)



Step 3: Grayscale

Concept: matrix product `@`; a dimension disappears

```
weights = np.array([0.299, 0.587, 0.114])
gray = img @ weights
# (H, W, 3) @ (3,) -> (H, W)
```

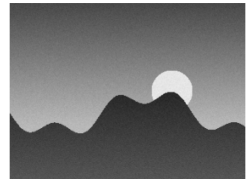
One line replaces a triple loop. Shape went 3D $\rightarrow$ 2D.

```
gray.shape = (240, 320)
gray.dtype = float64
min, max = 27.0, 240.4
```

color (H,W,3)



gray (H,W)



Step 4: Rescale to 0–1

Concept: element-wise arithmetic

```
gray = gray / 255.0
```

```
min, max = 0.11, 0.94

uint8 trap:
[250] + 10 -> [4]
```

values now in [0, 1]



Step 5: Crop

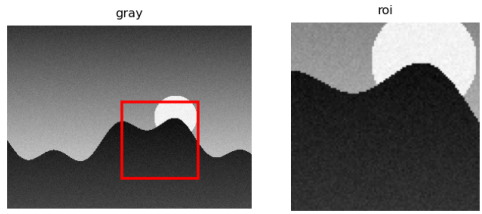
Concept: slicing `a[rows, cols]`; slices are *views*

```
roi = gray[100:200, 150:250]
print(roi.shape)
np.shares_memory(gray, roi)
np.shares_memory(gray, roi.copy())
```

```
roi.shape = (100, 100)

view shares memory with gray:
np.shares_memory(gray, roi) -> True
roi.copy() shares?     -> False
```

Modifying `roi` modifies `gray`. Use `.copy()` to avoid.



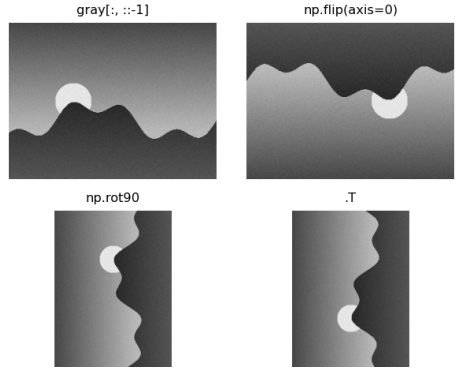
Step 6: Flip, rotate, transpose

Concept: negative strides, `np.flip`, `np.rot90`, `.T`; axis

```
mirrored = gray[:, ::-1]
upside   = np.flip(gray, axis=0)
rotated  = np.rot90(gray)
transposed = gray.T
```

```
gray.shape:       (240, 320)
mirrored.shape:   (240, 320)
rotated.shape:    (320, 240)
transposed.shape: (320, 240)
```

axis 0 runs down the rows, axis 1 across the columns.



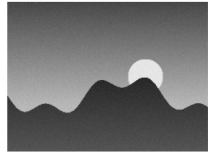
Step 7: Brightness and contrast

Concept: broadcasting scalars; `np.clip`

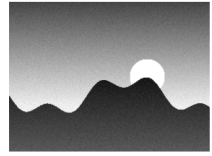
```
c, b = 1.5, 0.1
adjusted = np.clip(
    c * (gray - 0.5) + 0.5 + b, 0, 1)
```

```
before: min 0.11  max 0.94  mean 0.42
after:  min 0.01  max 1.00  mean 0.47
```

before



after



Step 8: Gamma correction

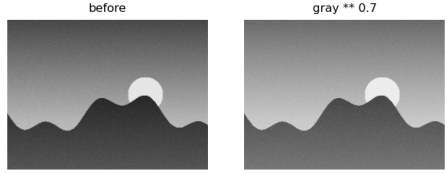
Concept: universal functions (ufuncs): `**`, `np.power`, `np.sqrt`, `np.exp`, `np.log`

```
gam = gray ** 0.7   # brightens midtones
```

Compare with the nested loops earlier.

```
mean before: 0.417
mean after:  0.534

same shape: True
```



Steps 9–10: Threshold and mask

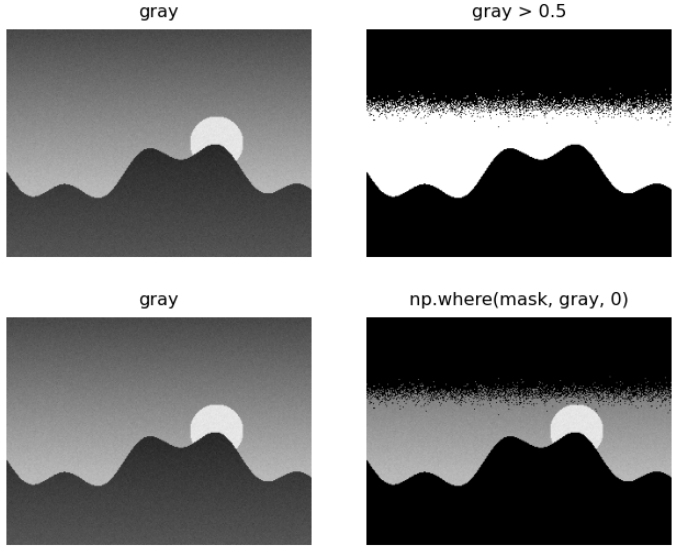
Concept: comparisons give boolean arrays; boolean indexing; `np.where`

```
mask = gray > 0.5
mask.mean()      # fraction True

hi = np.where(mask, gray, 0.0)
bright = gray[mask]
```

```
mask.dtype = bool
mask.shape = (240, 320)
fraction bright = 0.299
```

```
highlighted.shape = (240, 320)
bright_values.shape = (22997,)
   (1D: 2D structure lost)
```



Step 11: Downsample by averaging 2×2

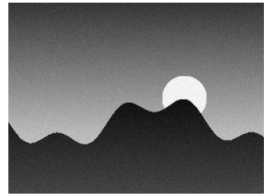
Concept: `reshape`; reducing over several axes

```
h, w = gray.shape
h2, w2 = h - h % 2, w - w % 2
blocks = gray[:h2, :w2].reshape(
    h2//2, 2, w2//2, 2)
small = blocks.mean(axis=(1, 3))
```

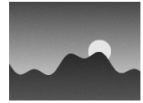
```
gray.shape   = (240, 320)
blocks.shape = (120, 2, 160, 2)
small.shape  = (120, 160)
```

Reshape doesn't move data; it changes how the same memory is *interpreted*.

(240, 320)



(120, 160) (drawn at same scale/2)



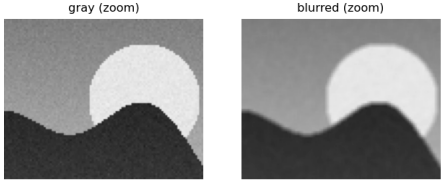
Step 12: Blur with a sliding window

Concept: `np.pad`, `sliding_window_view`, `sum(axis=...)`

```
kernel = np.ones((3, 3)) / 9
padded = np.pad(gray, 1, mode="edge")
windows = sliding_window_view(
    padded, (3, 3))        # (H, W, 3, 3)
blurred = (windows * kernel).sum(
    axis=(2, 3))
```

```
padded.shape  = (242, 322)
windows.shape = (240, 320, 3, 3)
blurred.shape = (240, 320)
windows is a view: True
```

This is convolution, the idea behind CNNs.



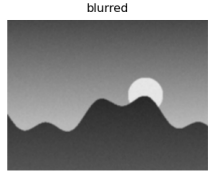
Step 13: Edge detection

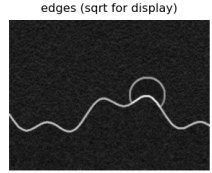
Concept: `np.gradient`, `np.hypot`

```
gy, gx = np.gradient(blurred)
edges = np.hypot(gx, gy)
edges = edges / edges.max()
```

```
gx.shape = (240, 320)
edges: min 0.00, max 1.00
```

Edges are where brightness changes fast: calculus on a grid.



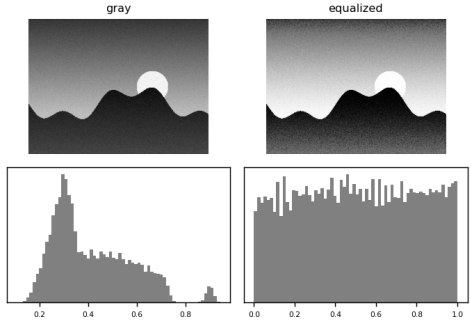


Step 14: Histogram and equalization

Concept: `np.histogram`, `np.cumsum`, `np.interp`

```
counts, bin_edges = np.histogram(
    gray, bins=256, range=(0, 1))
cdf = counts.cumsum()
cdf = cdf / cdf[-1]
equalized = np.interp(
    gray.ravel(), bin_edges[:-1], cdf
).reshape(gray.shape)
```

```
counts.shape    = (256,)
bin_edges.shape = (257,)
cdf[-1] = 1.0
std before 0.162, after 0.287
```

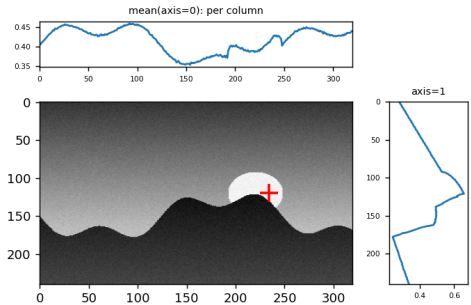


Step 15: Summary statistics

Concept: reductions: `mean std argmax unravel_index`; `axis` = the axis that gets *collapsed*

```
print(gray.mean(), gray.std())
gray.mean(axis=0).shape   # per column
gray.mean(axis=1).shape   # per row
idx = np.unravel_index(
    gray.argmax(), gray.shape)
```

```
mean, std = 0.417, 0.162
mean(axis=0).shape = (320,)
mean(axis=1).shape = (240,)
brightest (row, col) = (119, 234)
```



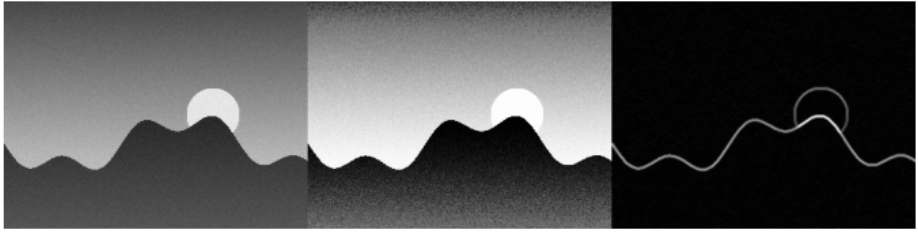
Steps 16–17: Reshape, stack, save

Concept: `ravel`, `hstack`, converting back to `uint8`

```
flat = gray.ravel()
side = np.hstack([gray, equalized, edges])
out = (np.clip(side, 0, 1) * 255
       ).astype(np.uint8)
Image.fromarray(out).save("result.png")
```

```
flat.shape = (76800,)
side.shape = (240, 960)
saved dtype = uint8
```

gray | equalized | edges



Summary: Numpy functions and capabilities

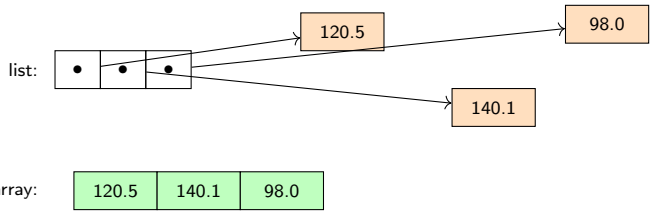
Step	Operation	NumPy concept
1–2	Load, inspect	`asarray`, `shape`, `dtype`, `ndim`
3	Grayscale	`@` / `dot`
4	Rescale	element-wise arithmetic
5	Crop	slicing (views)
6	Flip / rotate	negative strides, `flip`, `rot90`, `.T`
7	Contrast	broadcasting, `clip`
8	Gamma	ufuncs
9–10	Threshold, mask	boolean arrays, `where`
11	Downsample	`reshape`, multi-axis `mean`
12	Blur	`pad`, `sliding_window_view`, `sum(axis=...)`
13	Edges	`gradient`, `hypot`
14	Histogram	`histogram`, `cumsum`, `interp`
15	Statistics	reductions, `argmax`
16–17	Stack, save	`hstack`, `ravel`, `astype`

shape, axes, slicing, broadcasting, ufuncs, masks, reductions, reshaping cover the whole pipeline.

Numpy internals ...

Python's native model is a poor fit

```
import sys
xs = [1.0, 2.0, 3.0]
print(sys.getsizeof(xs))       # list: array of pointers
print(sys.getsizeof(xs[0]))    # each float: a boxed PyObject (24 B)
print([id(x) for x in xs])     # scattered on the heap
```



Pointer + boxed 24-byte object per number, scattered on the heap, with a type check per operation. NumPy: one contiguous block.

What a programmer wants from numeric data

- **Homogeneous**: one type for every element
- **Contiguous**: one block of memory
- **Fixed width**: 8 bytes per `float64`
- **Addressable by arithmetic**: $\text{addr}(i) = \text{base} + i \cdot \text{itemsize}$

This is the C array model. NumPy brings it to Python.

Memory is a flat sequence of bytes

```
import numpy as np
a = np.array([1.5, 2.5, 3.5])
print(a.nbytes, a.itemsize)     # 24, 8
print(a.tobytes()[:8])          # raw bytes of a[0]
print(a.ctypes.data)            # address of the buffer
```

Element i lives at a computed offset; no pointers to chase.

dtype is an interpretation of bytes

```
a = np.array([1.0], dtype=np.float64)
print(a.view(np.int64))          # [4607182418800017408]
print(a.view(np.uint8))          # the 8 raw bytes
b = np.arange(4, dtype=np.int32)
print(b.view(np.int16))          # same 16 bytes, twice as many items
```

Same bytes, different meaning. `astype` converts; `view` reinterprets.

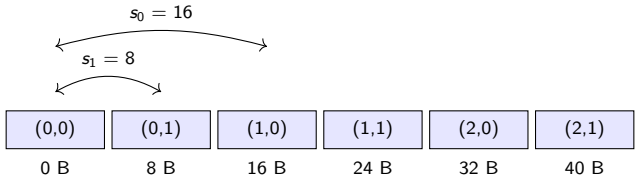
The four ingredients of an ndarray

```
a = np.arange(6, dtype=np.int32).reshape(2, 3)
print(a.data)      # 1. buffer
print(a.dtype)     # 2. int32
print(a.shape)     # 3. (2, 3)
print(a.strides)   # 4. (12, 4)  bytes to step per axis
```

Everything else (slicing, transpose, broadcasting) is manipulating shape and strides.

N-D indexing is stride arithmetic

$$\text{addr}(i, j) = \text{base} + i \cdot s_0 + j \cdot s_1$$




```
c = np.zeros((3, 4), order='C')   # row-major: strides (32, 8)
f = np.zeros((3, 4), order='F')   # column-major: strides (8, 24)
print(c.strides, f.strides)
```

Traverse along the smallest stride for cache-friendly access.

Views vs copies

```
a = np.arange(6).reshape(2, 3)
v = a[:, 1:]          # view: new shape/offset, same buffer
t = a.T               # view: strides swapped
print(t.strides, a.strides)
print(v.base is a, np.shares_memory(a, v))   # True True
v[0, 0] = 99          # a changes too!
c = a.copy()          # independent buffer
```